

Расширенный гайд по Docker для DevOps-специалистов

Docker стал ключевым инструментом в работе DevOps-инженеров, позволяя упаковывать и запускать приложения в изолированных контейнерах. В этом руководстве рассмотрены продвинутые аспекты использования Docker, от установки на разных ОС до best practices, безопасности и интеграции в CI/CD. Гайд предназначен для специалистов среднего и высокого уровня, уже знакомых с базовыми концепциями контейнеризации.

Установка и настройка Docker

Linux: На Linux Docker обычно устанавливается как Docker Engine (Community Edition). Для дистрибутивов Debian/Ubuntu и др. доступны пакеты в официальном репозитории Docker. Рекомендуется установка последней версии Docker Engine через менеджер пакетов с добавлением официального Docker-репозитория. Например, для Ubuntu: установка пакетов `docker-ce`, `docker-ce-cli` и `containerd.io`. После установки обязательно выполните пост-настройку: добавьте своего пользователя в группу `docker`, чтобы работать с Docker без `sudo` ¹ ². Это делается командой: `sudo usermod -aG docker $USER` с последующим выходом и входом в систему (или командой `newgrp docker`) ³ ⁴. Кроме того, убедитесь, что демон Docker запускается при старте системы (в современных дистрибутивах он обычно настраивается автоматически через `systemd`). После этого можно проверить успешность установки командой `docker run hello-world` ².

Windows: Для Windows и macOS Docker предоставляет **Docker Desktop** – удобный десктоп-пакет с встроенным Docker Engine. На Windows Docker Desktop требует включения подсистемы WSL2 или Hyper-V для запуска Linux-контейнеров. Предпочтительным бэкендом является WSL2, позволяющий использовать ядро Linux для контейнеров внутри Windows ⁵. Установите Docker Desktop с официального сайта, включите интеграцию с WSL2 в настройках. Docker Desktop на Windows позволяет запускать как Linux-контейнеры (через WSL2), так и Windows-контейнеры (необходима ОС Windows 10/11 Pro или Windows Server для поддержки контейнеров Windows). После установки Docker Desktop убедитесь, что включён режим использования WSL2-движка, а необходимые ресурсы (CPU, RAM) выделены в настройках, если вы планируете запускать ресурсоёмкие нагрузки.

macOS: Docker Desktop для Mac устанавливается из .dmg-пакета. Требуется современная версия macOS и поддержка гипервизора (Docker Desktop использует легковесную VM через `Hypervisor.framework`). После установки приложение Docker Desktop предоставит демона Docker. Имейте в виду, что на Mac и Windows Docker работает внутри виртуализированного Linux-окружения, поэтому производительность дисковых операций на маунтах из файловой системы хоста может быть ниже, чем на Linux-хосте. Для повышения производительности при разработке на Windows/macOS рекомендуется хранить файлы проекта на стороне Linux VM (для Windows – внутри WSL2 файловой системы) ⁶.

Проверка установки: после установки на любой системе выполните `docker version` и `docker info` – эти команды отобразят версию клиента/демона Docker и базовую информацию

о конфигурации. Убедитесь, что демон запущен (Docker Desktop должен быть в Running-состоянии). Если установка выполнена правильно, команда `docker run hello-world` должна скачать тестовый образ и запустить контейнер с выводом приветственного сообщения.

Примечание: Docker Desktop для Linux также доступен и объединяет схожую функциональность (графический интерфейс, интеграция с Docker Hub), однако на серверных Linux обычно достаточно установки Docker Engine. Docker Engine доступен для большинства популярных платформ (Ubuntu, Debian, CentOS, Fedora и др.) ⁷. Обновляйте Docker до актуальной версии для получения последних возможностей и исправлений безопасности.

Docker CLI: команды и практическое применение

Docker предоставляет обширный **CLI (command-line interface)** для управления контейнерами, образами, сетями, томами и другими объектами. Всего в Docker CLI доступно порядка 60 команд, которые можно условно разделить на несколько категорий ⁸:

- **Управление контейнерами:** запуск и остановка контейнеров, просмотр списка, подключение к работающим контейнерам и пр.
- **Управление образами:** сборка (build) и загрузка (pull/push) образов, просмотр локальных образов.
- **Управление данными:** операции с томами (volumes) и файловыми системами контейнеров.
- **Управление сетями:** создание и настройка виртуальных сетей Docker.
- **Общие команды:** информационные и служебные (версии, логи, очистка).

Ниже приводится обзор ключевых команд Docker CLI с примерами.

Управление образами

- `docker build`: собрать образ из Dockerfile. Пример:

```
docker build -t myapp:1.0 .
```

Эта команда читает Dockerfile в текущей директории (.) и создает образ с тегом `myapp:1.0`. Флаг `-t` задаёт имя:тег образа. Используйте многоступенчатую сборку и оптимизации кэша при создании образов (подробнее в разделе про Dockerfile). С флагом `--build-arg` можно передавать билд-аргументы, а `--target` позволяет собрать конкретный stage в многоступенчатом Dockerfile.

- `docker pull`: загрузить образ из Docker Registry (например, Docker Hub). Например, `docker pull nginx:latest` загрузит официальный образ nginx с тегом latest.
- `docker push`: отправить образ в регистр (репозиторий). Перед отправкой требуется выполнить `docker login` и убедиться, что образ имеет тег, соответствующий вашему репозиторию (например, `username/repo:tag`). Далее: `docker push username/repo:tag`.
- `docker images`: отобразить список локально сохранённых образов. Показывает репозиторное имя, тег, идентификатор образа и размер.

ШПАРАГЛКА ПО DOCKER CLI	
ОБРАЗЫ • <code>docker pull <image></code> • <code>docker images</code> • <code>docker build .</code> • <code>docker rmi <image></code>	КОНТЕЙНЕРЫ • <code>docker run <image></code> • Пуск контейнер • <code>docker ps</code> • Обзор running контейнер • <code>docker stop <container></code> • Лог о контейнере • <code>docker exec -it <container> sh</code> • Обеспечить команду • <code>docker rm <container></code> • Дать к контейнеру
КОНТЕЙНЕРЫ • <code>docker run <image></code> • Запуск контейнер • <code>docker ps</code> • Набор подисобытий • <code>docker stop <container></code> • Лог о контейнере • <code>docker logs <container></code> • Соединить контейнера • <code>docker exec -it <container> sh</code> • Обеспечить контейнер • <code>docker rm <container></code> • Дать к контейнер	ПРОЧЕЕ • <code>docker info</code> • Показать информация об оке • <code>docker compose up</code> • Запуск сервисов по docker-compose.yml • <code>docker volume ls</code> • Обзор несущей

- `docker rmi`: удалить образ по имени или ID. Можно применять для очистки неиспользуемых образов. Для массовой очистки используйте `docker image prune` (удалить dangling-образы) или `docker system prune` (удалить всё неиспользуемое).
- `docker tag`: присвоить образу новый тег (репозиторное имя). Например, `docker tag myapp:1.0 myregistry.example.com/project/myapp:1.0` – присвоит существующему образу новое имя для отправки в приватный регистр.

Управление контейнерами

- `docker run`: запустить новый контейнер из указанного образа. Это одна из самых часто используемых команд. Ключевые опции:
- `-d` (detached) – запустить контейнер в фоне (отсоединённо);
- `-p host_port:container_port` – опубликовать порт контейнера наружу (порт хоста);
- `-v /host/dir:/container/dir` – смонтировать директорию с хоста в контейнер (bind mount);
- `-v volume_name:/container/dir` – подключить именованный том;
- `--name` – задать имя контейнеру (удобно для управления);
- `--network` – подключить контейнер к определённой Docker-сети.

Пример:

```
docker run -d --name myweb -p 80:80 -v mydata:/app/data nginx:latest
```

Эта команда запустит контейнер `myweb` на основе образа `nginx`, опубликует порт 80 контейнера на порт 80 хоста, подключит том `mydata` в каталог `/app/data` внутри контейнера. Контейнер будет работать в фоне благодаря `-d`.

- `docker ps`: список работающих контейнеров. С флагом `-a` показывает все контейнеры (включая остановленные). Полезные опции: `-q` (выводит только ID контейнеров), `--filter` для фильтрации по состоянию, имени и пр.
- `docker stop` / `docker start`: остановить или запустить существующий контейнер. Остановка посылает SIGTERM, а через таймаут SIGKILL. Команда `docker restart` перезапускает контейнер (эквивалент stop + start).
- `docker kill`: немедленно завершить контейнер посылкой SIGKILL (используется в экстренных случаях, т.к. не даёт приложению корректно завершиться).
- `docker rm`: удалить остановленный контейнер. Обычно сначала останавливают, затем удаляют. Флаг `-f` позволяет форсированно удалить работающий контейнер (с предварительной посылкой SIGKILL).
- `docker logs`: показать логи контейнера (вывод stdout/stderr приложения). Часто используется с `-f` (follow) для потока логов в реальном времени, и `--tail N` чтобы показать последние N строк. Например: `docker logs -f --tail 50 myweb`.
- `docker exec`: выполнить команду внутри работающего контейнера. Необходим для отладки или управления. Ключевая опция `-it` (интерактивный режим с терминалом).

Например: `docker exec -it myweb /bin/bash` откроет интерактивную оболочку Bash внутри контейнера `myweb`. Эта команда часто используется для "входа" в контейнер.

- `docker inspect`: получить детальную информацию о контейнере или образе в формате JSON (конфигурация, смонтированные тома, сетевые настройки и т.д.). Пример: `docker inspect myweb` вернёт JSON, который можно отфильтровать с помощью `--format` (используя шаблоны Go) для вывода конкретных полей.
- `docker cp`: копировать файлы между контейнером и хостом. Например, `docker cp myweb:/var/log/nginx/error.log ./error.log` извлечёт файл логов nginx из контейнера на хост.
- `docker attach`: присоединиться к интерактивному потоку запущенного (detached) контейнера, чтобы увидеть его вывод или взаимодействовать. (Важно: при отсоединении сочетанием клавиш Ctrl+P Ctrl+Q контейнер продолжит работу.)

Сети Docker

- `docker network ls`: показать список Docker-сетей (bridge, host, none и пользовательские).
- `docker network create`: создать новую пользовательскую сеть. Например, `docker network create backend-net` создаст сеть с именем *backend-net* с драйвером по умолчанию (bridge). Можно указать `-d overlay` для сети оверлей (требуется Swarm режим), или другие драйверы (macvlan и т.д.), а также подсеть `--subnet` и шлюз `--gateway` при необходимости.
- `docker network inspect`: просмотреть подробности сети, включая подключённые контейнеры и конфигурацию подсети.
- `docker network connect` / `disconnect`: подключить или отключить работающий контейнер к пользовательской сети. Это удобно для изменения топологии сети без перезапуска контейнера.
- При запуске `docker run` флагом `--network` можно сразу указать, к какой сети подключить контейнер. Пользовательские bridge-сети позволяют контейнерам обращаться друг к другу по имени (через встроенный DNS). Например, если два контейнера запущены в одной сети *backend-net*, то контейнер *app* сможет достигаться до контейнера *db* по хостнейму *db*.

Том и хранилище

- `docker volume create`: создать именованный том для хранения данных. Пример: `docker volume create dbdata`.
- `docker volume ls`: список томов. Именованные тома хранятся Docker'ом обычно в `/var/lib/docker/volumes/` на хосте.
- `docker volume inspect`: подробности тома (путь на хосте и др.).

- `docker volume rm`: удалить том (если он больше не используется контейнерами).
Осторожно: данные на томе будут удалены.
- `docker volume prune`: удалить все тома, не привязанные к контейнерам (используется для очистки).
- Кроме именованных томов, Docker позволяет использовать *bind mounts* (прямое монтирование папки хоста) и *tmpfs*. Для `docker run` современный синтаксис `--mount` часто предпочтителен, так как более выразителен. Например:
 - `--mount type=volume,source=dbdata,target=/var/lib/mysql`
 - `--mount type=bind,source="$(pwd)"/app,target=/app`
 - `--mount type=tmpfs,target=/tmp,tmpfs-size=100m` (создаст tmpfs размером 100 МБ, смонтированный в /tmp контейнера).

Прочие команды

- `docker login` / `logout`: аутентификация на Docker Registry (Docker Hub или другом). Требуется перед `push` к приватным репозиториям.
- `docker search`: поиск образов на Docker Hub из CLI.
- `docker stats`: мониторинг потребления ресурсов запущенных контейнеров (CPU, память, сеть, диск) в режиме top.
- `docker top`: показать процессы внутри контейнера.
- `docker history`: показать историю сборки образа (последовательность слоёв/инструкций Dockerfile).
- `docker export` / `docker import`: выгрузить файловую систему контейнера в tar-архив и импортировать её обратно как образ (менее распространено, но полезно для бэкапов без Dockerfile).
- `docker system df`: показать суммарное использование диска образами, контейнерами, томами.
- `docker system prune`: **удалить все** остановленные контейнеры, неиспользуемые сети, dangling-образы (без тегов) и кеш сборки. Полезно для очистки, но использовать с осторожностью на продакшене.

Эффективное владение CLI Docker позволяет автоматизировать множество задач, быстро управлять ресурсами и разбираться в состоянии контейнерной среды. Полный список команд доступен в справке (`docker --help`) и официальной документации Docker ⁸, а также в виде шпаргалок.

Docker Compose: структура, примеры, продвинутые возможности

Docker Compose – инструмент для определения и запуска многоконтейнерных приложений. Он использует файл конфигурации (по умолчанию `docker-compose.yml` или `compose.yaml`), в котором описываются сервисы (контейнеры), их образы, параметры запуска, сети, тома и прочие настройки. Compose значительно упрощает управление связанной группой контейнеров (например, приложение + база данных + кэш) единым командованием.

Базовая структура `docker-compose.yml`

Compose-файл пишется в формате YAML. Простейший пример для веб-приложения с базой данных:

```
version: '3.9'
services:
  web:
    build: .
    ports:
      - "8000:8000"
    depends_on:
      - db
  db:
    image: postgres:15-alpine
    volumes:
      - db_data:/var/lib/postgresql/data
volumes:
  db_data:
```

В этом примере определены два сервиса: **web** (строится из Dockerfile в текущей директории) и **db** (образ Postgres). Сервис *web* публикует порт 8000 и зависит от *db* (Compose гарантирует, что контейнер *db* будет запущен до *web* ⁹, хотя полного ожидания готовности без дополнительных настроек не происходит – об этом далее). Том *db_data* будет создан для сохранения данных БД и подключён к контейнеру *db*.

Чтобы запустить эту композицию, достаточно выполнить `docker-compose up -d` (используя утилиту `docker-compose`) или `docker compose up -d` (если используется встроенная Compose V2). Флаг `-d` запустит все контейнеры в фоновом режиме. Команда автоматически создаст сеть по умолчанию для этой композиции (имя сети обычно основано на имени проекта/директории) и подключит оба контейнера к ней, позволяя *web* обращаться к *db* по имени хоста "db".

Расширенные возможности Compose

Контроль порядка запуска и готовности: По умолчанию Compose запускает контейнеры в порядке, определяемом `depends_on`, но **не ждёт** готовности сервисов (только факт старта) ⁹. Для управления зависимостями по здоровью можно использовать `healthcheck`'и и условие `condition: service_healthy` (Compose v3) ¹⁰. Например, настроив в сервисе *db* `healthcheck`, можно указать для *web* `depends_on: db: condition: service_healthy` (в Docker Compose v3.9+ это поддерживается). Либо использовать внешние скрипты ожидания

(например, популярный скрипт `wait-for-it.sh` или утилиту `dockerize`), вызываемые в `entrypoint` контейнера ¹¹ ¹². В продакшене для гарантий обычно полагаются на оркестраторы (Swarm, Kubernetes) или ретрай-логики на уровне приложения, но для dev/test среды Compose может управлять порядком запуска.

Переменные окружения и .env: Compose поддерживает внедрение переменных окружения. Например, вместо хардкода порта или версии образа можно использовать синтаксис `${VARIABLE}` в YAML. Значения берутся либо из текущих переменных окружения Shell, либо из файла `.env` в той же директории ¹³ ¹⁴. Файл `.env` автоматически загружается и может содержать пары ключ=значение. Переменные из ENV имеют приоритет над `.env`-файлом, если заданы обоими способами ¹⁵. Пример: можно вынести тег образа в `.env`: `TAG=15-alpine`, а в `compose.yml` указать `image: postgres:${TAG}` ¹⁶ ¹⁷ – при запуске подставится значение `15-alpine`. Это облегчает перенос одной и той же композиции между средами (dev/staging/prod), изменяя лишь переменные.

Несколько файлов Compose и профили: Для разных сред можно переопределять часть настроек, используя несколько YAML-файлов. Например, базовый `docker-compose.yml` и дополнительный `docker-compose.prod.yml` с продакшен-настройками. При запуске указывается несколько файлов флагом `-f` (сначала базовый, потом override):

```
docker-compose -f docker-compose.yml -f docker-compose.prod.yml up -d
```

Docker Compose объединит конфигурации, причём параметры из второго файла переопределят или дополнят базовый ¹⁸ ¹⁹. Это удобно для настройки, например, разных ресурсов или переменных для продакшена без дублирования всего файла. В новых версиях Compose также есть концепция *профилей* (profiles) – можно в одном файле пометить сервисы определённым профилем и включать/выключать их через переменную окружения `COMPOSE_PROFILES`, либо ключ `--profile`.

Масштабирование сервисов: Команда `docker-compose up --scale <service>=N` позволяет запустить несколько экземпляров одного сервиса (только в режиме без зависимости от уникальных ресурсов, например масштабировать фронтенд). В Compose v3 для Swarm есть секция `deploy: replicas: N`, но в обычном локальном режиме она не применяется. Тем не менее, `docker-compose up -d --scale web=3` запустит 3 контейнера сервиса `web`, которые будут балансироваться DNS round-robin'ом внутри сети Compose.

Запуск нескольких копий проекта: Если нужно поднять несколько изолированных копий одной и той же композиции (например, для параллельных тестов), используйте флаг `-p/--project-name` для задания имени проекта. По умолчанию именем проекта является название директории, но `docker-compose -p test1 up -d` и `docker-compose -p test2 up -d` запустят два независимых набора контейнеров, томов и сетей, даже если Compose-файлы одинаковые ²⁰. Это полезно для изоляции.

Расширение сервисов (extends): В YAML Compose v3 поддерживается ключ `x-*` для определения «снippetов» конфигурации, а также директива `extends` (в версии 3.8 и ниже) для наследования конфигурации одного сервиса от другого или из внешнего файла. Однако `extends` накладывает ограничения – не все поля переносятся (например, `links`, `volumes_from` не наследуются) ²¹. В практике сейчас чаще дублируют нужные поля или используют несколько файлов, чем `extends`.

Дополнительные возможности: Compose позволяет задавать **healthcheck** для сервисов (ключ `healthcheck` внутри описания сервиса) – Docker будет отслеживать состояние контейнера. Можно задавать рестарт-политику `restart: "always"` (или `on-failure`, `no`, `unless-stopped`) для автоперезапуска упавших контейнеров. Секцией `deploy` (Swarm) можно описать лимиты ресурсов, правила размещения – хотя они применяются только в Swarm-моде, их наличие не влияет на локальный запуск. Через Compose можно подключать **секреты и конфиги** (тоже в Swarm-моде или с Docker Desktop и Compose v2 – например, `docker compose config` и `docker compose secret` позволяют работать с Secrets в локальном контексте Docker Desktop).

В продакшене Docker Compose часто используют для локальной разработки и тестирования. В боевой эксплуатации же либо переходят на Docker Swarm (с использованием `docker stack deploy` тех же Compose-файлов), либо на Kubernetes (для чего Compose-файлы могут конвертироваться или переписываться на Helm/manifest). Тем не менее, знание Docker Compose полезно: оно облегчает локальный цикл разработки, позволяет эмулировать сложные многосервисные окружения и быстро запускать/останавливать весь стек приложений одной командой.

Использование Docker в CI/CD-пайплайнах

Контейнеры Docker отлично интегрируются в процессы CI/CD. Они позволяют изолировать среду сборки, упаковать артефакты в образы и деплоить эти образы на различные окружения. Рассмотрим, как Docker применяется на популярных платформах CI/CD, таких как GitLab CI и GitHub Actions, а также общие подходы.

GitLab CI/CD

В GitLab CI есть несколько способов использования Docker в заданиях (jobs) конвейера:

- **Запуск задач внутри Docker-контейнера:** GitLab Runner поддерживает *Docker executor*, позволяющий запускать каждый CI job внутри контейнера. В `.gitlab-ci.yml` можно указать образ для job (`image: python:3.11-alpine` например), и код будет выполняться внутри этого контейнера. Это удобно для изоляции окружения сборки.
- **Сборка Docker-образов внутри CI (Docker-in-Docker):** Для того чтобы внутри CI job запускать команды `docker build`, runner должен иметь доступ к Docker-демону. Один вариант – включить *privileged mode* и использовать образ `docker:dind` (Docker-in-Docker) в качестве сервиса. Пример `.gitlab-ci.yml` для сборки образа:

```
build_image:
  image: docker:24.0.5      # образ с установленным Docker CLI
  services:
    - docker:24.0.5-dind    # сервис: демон Docker в контейнере
  variables:
    DOCKER_DRIVER: overlay2  # драйвер хранилища для DinD
  script:
    - docker info            # проверка доступа к демону
    - docker build -t myregistry.example.com/app:${CI_COMMIT_SHA} .
    - docker push myregistry.example.com/app:${CI_COMMIT_SHA}
```

Здесь job запускается в контейнере `docker:24.0.5` (содержит Docker CLI). Параллельно поднимается сервис `docker:24.0.5-dind` – Docker демон в режиме DinD. Runner должен быть запущен с `--privileged`, чтобы DinD работал ²² ²³. Переменная `DOCKER_DRIVER` установлена в `overlay2` для лучшей производительности. Внутри скрипта выполняются обычные команды `docker build / push`. GitLab обеспечит, что Docker-клиент из основного контейнера обращается к Docker-демону сервиса через переменные окружения (обычно `Docker host tcp://docker:2375` без TLS по умолчанию, или `DOCKER_TLS_CERTDIR` для TLS). **Важно:** этот способ, хотя и удобен, несёт риски, так как запущенный в privileged-режиме DinD даёт очень широкие права (по сути, root-доступ внутри Runner) ²² ²⁴. Альтернатива – *Docker socket binding* (примонтировать `/var/run/docker.sock` хоста) либо *канико/Buildah* для сборки образов без привилегий.

- **Docker socket binding:** Если GitLab Runner установлен на Linux-хосте, где уже есть Docker, можно зарегистрировать Runner с `shell` executor. Тогда job будет выполняться напрямую на хосте, и имеющийся Docker демон сможет выполнять команды (достаточно дать пользователю Runner права в группе docker) ²⁵. Либо, используя Docker executor, но вместо DinD примонтировать внутрь контейнера сокет хоста:

```
image: docker:24.0.5
services: []
variables:
  DOCKER_HOST: "tcp://docker:2375"
```

и запустить сам Docker daemon на хосте или как сервис. Этот способ сложнее из-за прав и безопасности (фактически контейнер job'a управляет хостовым Docker).

В итоге, для GitLab CI наиболее распространённый подход – использовать образ `docker:latest` + сервис `docker:dind`, либо установить Runner на собственный сервер и использовать shell-режим. В обоих случаях, получив образ, имеет смысл загрузить его в GitLab Container Registry (если используется GitLab) или иной регистр, а затем деплоить с помощью образа.

GitLab CI также поддерживает *артефакты* и *кэш*, которые можно применять совместно с Docker. Например, можно кэшировать директорию сборки между запусками, чтобы ускорить `docker build` (репозиторий кэша слоёв, хотя полный кеш слоёв Docker в GitLab CI настроить непросто без специальных driver'ов). Интеграция со SAST/DAST: есть возможность встроить сканирование образов (например, с Trivy или [Docker Scan \(Snyk\)](#) в pipeline для проверки уязвимостей.

GitHub Actions

В GitHub Actions также легко запускать Docker-команды, так как runner (например, `ubuntu-latest`) уже включает Docker Engine. Workflows (файлы `.github/workflows/*.yaml`) позволяют:

- **Собрать и отправить образ:** Использовать готовые actions. Например, официальный набор от Docker: `docker/login-action`, `docker/build-push-action` ²⁶. Пример фрагмента workflow:

```
jobs:
  build:
```

```

runs-on: ubuntu-latest
steps:
  - uses: actions/checkout@v3
  - uses: docker/setup-buildx-action@v2    # настроить builder (BuildKit)
  - uses: docker/login-action@v2
    with:
      registry: ghcr.io
      username: ${ secrets.CR_USERNAME }
      password: ${ secrets.CR_PAT }
  - uses: docker/build-push-action@v4
    with:
      context: .
      push: true
      tags: ghcr.io/myorg/myapp:${ github.sha }
      labels: type=oci

```

Здесь выполняется логин в GitHub Container Registry (ghcr.io) с использованием секретов репозитория, а затем сборка и пуш. Action `docker/build-push-action` под капотом использует BuildKit (через `docker buildx`) для кроссплатформенности и кэширования. Эта высокоуровневая декларация упрощает процесс.

- **Запуск сервисов для тестов:** GitHub Actions поддерживает ключ `services` для шагов, позволяя поднять временный контейнер БД или другого сервиса для интеграционных тестов. Например:

```

services:
  db:
    image: postgres:15-alpine
    env:
      POSTGRES_PASSWORD: example
    ports:
      - 5432:5432

```

После этого внутри шага тестирования приложение сможет подключаться к `localhost:5432` (или к хосту `db` на порту 5432, в зависимости от сетевых настроек), где запущен контейнер Postgres.

- **Использование Docker Container Actions:** В GH Actions можно писать сами действия в виде Docker-контейнеров (с собственным Dockerfile). Тогда GitHub при выполнении workflow запустит указанный контейнер. Этот подход используется для изоляции действия и воспроизводимости, однако для большинства задач проще писать действия на JS или использовать готовые.
- **Сборка без привилегий:** В среде GitHub Actions нет необходимости поднимать DinD – runner уже может запускать контейнеры. Но если требуется полностью изолированная сборка, можно запускать self-hosted runner с Docker и использовать аналог DinD. Чаще всего, однако, достаточно команд `docker build` / `docker push` прямо в шаге (после `docker login`).

- **Пример деплоя:** После сборки образа можно, например, задействовать SSH-соединение к серверу для деплоя контейнера или обновления сервисов (если не используется оркестратор). Либо, если деплой на Kubernetes, загрузить образ в регистр и задействовать kubectl/helm для обновления.

Другие платформы CI/CD: В Jenkins Docker используется либо через плагин *Docker Pipeline* (декларативно запускать шаги в контейнерах или запускать DinD-агенты), либо просто выполнением shell-команд на агенте с установленным Docker. В GitLab и GitHub, как рассмотрено, есть встроенная поддержка. В Bitbucket Pipelines также Docker предустановлен, и сборка образов выполняется схожим образом (с YAML конфигурацией).

Лучшие практики в CI с Docker:

- Минимизировать привилегированные операции. По возможности использовать BuildKit с монтированием сокета вместо полного DinD, или специализированные инструменты (Kaniko, Buildah) для сборки образов в user-space.
- Очищать за собой: после сборки можно выполнять `docker image prune -af` в CI, чтобы не накапливать лишние образы на агентах (если runner не сбрасывает состояние сам).
- Использовать кеширование слоёв: например, в GitHub Actions `actions/cache` для кеша `/tmp/.buildx-cache` (эксперимент BuildKit cache) – это ускорит повторные билды.
- Сканировать образы прямо в pipeline (см. раздел "Безопасность"): интегрировать шаг с Trivy или Dockle, чтобы проваливать сборку при нахождении критических уязвимостей или нарушений политики. Это соответствует подходу "shift left" – обеспечение безопасности на этапе билда ²⁷.
- Хранить артефакты: помимо пуша образа, можно сохранить образ как файл (`docker save`) артефактом pipeline, но обычно проще публиковать в registry.

Лучшая практика написания Dockerfile

Написание Dockerfile – искусство, влияющее на размер, безопасность и производительность образа. Рассмотрим лучшие практики:

- **Минимизируйте базовый образ и количество слоёв:** Выбирайте лёгкие базовые образы, например Alpine Linux или узкоспециализированные образы. Чем меньше ненужных пакетов, тем меньше поверхность атаки и размер. Например, вместо `python:3.11` (на Debian) можно использовать `python:3.11-alpine` если совместимо. В пределе – `scratch` (пустой образ) для Go/Rust/C++ приложений, которые можно статически слинковать. Каждый слой (инструкция RUN, COPY etc.) добавляет размер, поэтому старайтесь сочетать команды. Однако балансируйте с читаемостью: 2-3 команды в RUN, объединённые с `&&` – нормально, но не пытайтесь слить всё в одну команду.
- **Многоступенчатая сборка (multi-stage builds):** Эта возможность Dockerfile позволяет сначала собрать/скомпилировать приложение в одном контейнере, а затем взять только необходимые артефакты в финальный образ. В первом FROM (build-stage) можно использовать тяжёлый образ с SDK, компиляторами, а во втором FROM – минимальный рантайм. Пример:

```
FROM golang:1.20 AS build
WORKDIR /src
COPY . .
```

```
RUN CGO_ENABLED=0 go build -o /app/myservice

FROM alpine:3.18 AS final
COPY --from=build /app/myservice /usr/local/bin/myservice
CMD ["myservice"]
```

Здесь итоговый образ основан на Alpine и содержит только бинарник, собранный на этапе build. Всё лишнее (компилятор, кеши) не попадает в финальный образ. Многоступенчатая сборка помогает радикально уменьшить размер образа и убрать секреты сборки. Docker при сборке пропускает неиспользуемые этапы, что тоже ускоряет процесс, если указать `--target` для конкретного этапа. Эта техника официально рекомендована ²⁹ и используется во многих официальных образах.

- **Оптимизация кеширования слоёв:** Docker кеширует слои, поэтому порядок инструкций влияет на возможность использовать кеш при повторных сборках. Правило: меняющиеся части (например, копирование исходников приложения) должны быть как можно ниже в Dockerfile. Например, сначала COPY только зависимостей (например, `package.json`) и `RUN npm ci` – это закешируется, пока `package.json` не изменился. Затем COPY всего кода и `RUN npm build` – если код меняется часто, только эти шаги будут пересобираются. Используйте `.dockerignore` файл, чтобы исключить из контекста сборки лишние файлы (git-репозиторий, артефакты, секреты), иначе любое изменение в них будет ломать кеш.
- **Удаление временных файлов и пакетов:** При установке пакетов (apt, apk, npm и т.п.) старайтесь удалять кеши и временные файлы в рамках того же RUN. Пример для apt: `RUN apt-get update && apt-get install -y gcc make ... && apt-get clean && rm -rf /var/lib/apt/lists/*`. Это предотвратит попадание ненужных файлов в слой образа. Аналогично, очищайте кеш менеджеров пакетов языков, если он не нужен runtime.
- **Не храните чувствительные данные в образе:** Никогда не жестко-кодируйте секреты, пароли, ключи в Dockerfile (даже если потом удаляете – они попадут в слои). Если необходимо скопировать конфиденциальный файл, делайте это **после** запуска контейнера (через volume или secrets). Dockerfile рекомендуется публиковать без секретов. Для сборки с секретами используйте возможности BuildKit: флаг `--secret` при `docker build` (пример: `docker build --secret id=mysecret,src=secret.txt ...`) и соответствующую инструкцию в Dockerfile `RUN --mount=type=secret,id=mysecret ...` – это позволит использовать секрет на этапе сборки, не сохраняя его в слое (BuildKit смонтирует файл временно) ³⁰ ³¹.
- **USER вместо root:** По умолчанию процессы в контейнере запускаются от root (UID 0), что может представлять риск. Лучшая практика – создавать несистемного пользователя внутри Dockerfile и переключаться на него перед запуском приложения (`USER appuser`) ³⁰ ³¹. При этом нужно убедиться, что у пользователя есть нужные права на файловую систему (папки, в которые пишется лог или данные) ³². Например, для Alpine: `RUN adduser -D -h /app appuser && chown -R appuser /app` затем `USER appuser`. Запуск от non-root пользователя минимизирует потенциальный ущерб от захваченного контейнера. Согласно исследованиям, ~58% образов запускаются под root, хотя в большинстве случаев это не требуется ³³.
- **Минимальные привилегии и возможности:** Связано с предыдущим – контейнеру не нужны лишние Linux-способности (capabilities). По умолчанию Docker отсекает часть

возможностей, но можно явно убрать и остальные, кроме необходимых: `docker run --cap-drop=ALL --cap-add=NET_BIND_SERVICE ...` (например, только возможность слушать порт <1024) ³⁴. Если приложение не требует привилегированных операций, запускайте контейнер без `--privileged` и с опцией `--security-opt no-new-privileges` ³⁵ для предотвращения повышения привилегий внутри контейнера.

- **Работа с файлами и правами:** Сделайте исполняемые файлы приложения не модифицируемыми (только для чтения для пользователя). Например, после копирования бинарника от root, переключившись на USER, у non-root не будет права изменить исполняемый файл – это повышает Immutable инфраструктуру (приложение не может самоизмениться) ³⁶. Если ваш USER не фиксирован (например, OpenShift случайно присваивает UID), избегайте жестких UID. Пишите в директории `/tmp` или делайте их world-writable при сборке, если нужно (так, чтобы любой UID мог писать) ³⁷ ³⁸.
- **Label и метаданные:** Добавляйте метки (`LABEL`) в Dockerfile – например, информацию о версии, описании, владельце. Это помогает в управлении образами. Стандарт – использовать формат меток [Label Schema](#) или OCI Image Spec annotations (например, `org.opencontainers.image.source=<URL>`). Метки не влияют на функциональность, но полезны для автоматизации (к примеру, указание git SHA, версии приложения и т.п.).
- **Инструкция ENTRYPOINT vs CMD:** Предпочтительно задавать ENTRYPOINT для основного исполняемого файла, а CMD для аргументов по умолчанию. Тогда можно запускать контейнер с разными аргументами. Например:

```
ENTRYPOINT ["java", "-jar", "/app/app.jar"]
CMD ["--server.port=8080"]
```

ENTRYPOINT задаёт базовую команду, CMD – параметры по умолчанию (которые можно переопределить при `docker run`). Или использовать только CMD для простоты, если не нужна жёсткая фиксация entrypoint.

- **Здоровье контейнера:** Можно прописать `HEALTHCHECK` в Dockerfile, чтобы Docker отслеживал, что контейнер работает правильно. Например:

```
HEALTHCHECK CMD curl -f http://localhost:8080/health || exit 1
```

Это запускает каждые N секунд команду и помечает контейнер как unhealthy при неудаче. В Compose и Swarm эти статусы могут учитываться (в Compose через `condition: service_healthy` как обсуждалось).

- **Linters и проверка Dockerfile:** Используйте инструменты вроде **Hadolint** – это линтер Dockerfile, который указывает на нарушения лучших практик (например, отсутствие `yum clean`, использование latest-тегов, отсутствие USER и др.). Hadolint можно интегрировать в CI и IDE. Еще инструмент – **WhaleLint** или **dockerfilelint**. Они помогут соблюдать описанные рекомендации.

Например, Hadolint способен выявить, если вы запускаете под root, не очистили менеджер пакетов или не зафиксировали версии пакетов. Инструмент **Dockle** работает уже по готовому

образу, проверяя соответствие образа рекомендациям (он анализирует образ на наличие паролей по умолчанию, пакетов в уязвимых версиях, отсутствие certs и т.п.) ³⁹ ⁴⁰ . Рекомендуется применять эти инструменты для автоматического аудита ваших Dockerfile и образов.

Выполнение этих best practices позволит создавать образы, оптимальные по размеру, быстро собирающиеся и с меньшим числом уязвимостей. Как отмечает руководство от Sysdig, хорошо сконструированный Dockerfile снижает риски ещё на этапе билда, экономя усилия на устранение проблем позже ⁴¹ ⁴² .

Безопасность: уязвимости, сканирование и секреты

Контейнеры Docker изолируют приложение, но неправильная конфигурация может привести к инцидентам. Рассмотрим меры по укреплению безопасности контейнеров и образов:

1. Актуальность и патчи: Держите обновлёнными как хостовую систему, так и Docker Engine ⁴³ . Уязвимости в ядре хоста (которые разделяют контейнеры) могут приводить к побегу из контейнера. Классический пример – уязвимость Dirty COW позволяла привилегии на хосте из контейнера при старом ядре ⁴⁴ . То же касается и самого Docker – обновления закрывают дыры. Регулярно обновляйте базовые образы ваших приложений до последних патчей (например, обновление Alpine или Ubuntu base image, когда в них закрыты CVE). Используйте автоматизированные проверки обновлений образов.

2. Минимизация поверхности атаки: Применяйте правило наименьших привилегий. Контейнер должен иметь только необходимое. Удалите из образа дебаг-инструменты, компиляторы, пакеты, не нужные во время выполнения. Чем меньше бинарей и библиотек, тем меньше потенциальных уязвимостей. Это связано с выбором минимального базового образа (обсуждалось выше) ⁴⁵ . Если приложению не нужен доступ к Интернету, запускайте контейнер в изолированной сети без внешнего доступа. Если не нужен доступ к Docker socket – ни в коем случае не монтируйте `/var/run/docker.sock` внутрь (это фактически даёт root-доступ к хосту) ⁴⁶ . Также избегайте `--privileged` флага и избыточных `--cap-add`: по умолчанию Docker уже урезает возможности, добавляйте только необходимые конкретно вашему приложению способности ядра ³⁴ .

3. Запуск в rootless-режиме: Новый подход – **Rootless Docker** (не от имени root на хосте). Docker может быть сконфигурирован так, что и демон, и контейнеры работают от обычного пользователя, используя user namespaces ⁴⁷ . Это сильно повышает безопасность: даже если контейнер вырвется, у процесса нет root-доступа на хосте. Rootless Docker сейчас поддерживается (хотя есть ограничения, например, ряд сетевых возможностей отсутствует). Рассмотрите использование rootless в средах, где максимальная изоляция важнее, чем гибкость сети.

4. Сканирование образов на уязвимости: Встраивайте сканирование в процесс разработки. Инструменты: **Trivy** (Aqua Security) – простой и быстрый сканер, проверяющий образы на известные уязвимости (CVE) и конфигурационные проблемы ²⁸ ⁴⁸ . Он имеет встроенную базу данных для основных дистрибутивов и языковых пакетов, и в секунды выдаёт отчёт. Например, `trivy image myapp:1.0` перечислит CVE для всех слоёв образа. Trivy поддерживает разные форматы вывода (таблица, JSON) и легко интегрируется в CI ⁴⁹ ⁵⁰ . Кроме уязвимостей, Trivy может обнаруживать встроенные секреты и чувствительные данные. Другой инструмент – **Anchore/Grype, Clair** (от Quay) или комьюнити-сканер *Dockle* и *Snyk*. **Dockle**, например, фокусируется на конфигурации образа: проверяет наличие USER, healthcheck, последних обновлений пакетов, отсутствие root-паролей, правильные разрешения файлов и т.п. ³⁹ ⁵¹ . Эти

инструменты можно запускать на этапе CI, чтобы предотвращать продвижение уязвимого образа по конвейеру ⁵² ⁵³ .

Docker Hub (Docker Scout) также предоставляет базовое сканирование (на базе Snyk) – если вы пушите в публичный репо, Docker Hub отметит уязвимости. Но для частных часто удобнее свои инструменты.

5. Управление секретами: Никогда не храните секреты (ключи, пароли) в образе или в открытом виде в переменных окружения, закоммитенных в VCS. Для Docker есть механизм **Docker Secrets**, но он доступен в Swarm/Kubernetes средах. В локальном Docker Compose можно имитировать secrets через файлы и монтирование. Лучшие практики: передавать секреты в контейнер через переменные окружения *только* во время запуска (например, через CI/CD, Kubernetes Secrets) и не сохранять их в Dockerfile. Если используете Compose для разработки, можно хранить секреты в отдельном .env файле (не кладя его в гит). В продакшне же – или Docker Swarm mode: `docker secret create` + `docker service create --secret`, или Kubernetes secrets, или внешние секретные хранилища (Vault, AWS Secrets Manager). Также убедитесь, что логи приложения не печатают секреты.

6. Изоляция и ограничения: Используйте лимиты ресурсов: `docker run -m 512m --cpus=1.0 ...` ограничит контейнер 512 МБ памяти и 1 ядром. Это предотвратит исчерпание ресурсов хоста при сбоях приложения (например, утечке памяти). Ограничьте число процессов (`--pids-limit`), дескрипторов, объем tmpfs, если контейнер не должен их превышать ⁵⁴ . Эти ограничения снижают влияние потенциального взломанного контейнера на систему.

Для данных: запускайте контейнеры с `--read-only` корневой FS, если приложению не нужно ничего писать кроме выносных томов ⁵⁵ . FS readonly + перечисление конкретных точек монтирования для записи (tmpfs для /tmp, именованный том для /data) – сильно затрудняет изменение файлов злоумышленником.

7. AppArmor/SELinux профили: Docker по умолчанию запускает контейнеры под профилем seccomp (отсекая опасные системные вызовы) и, если включен AppArmor, под профилем `docker-default`. Обычно этого достаточно. Но при повышенных требованиях безопасности можно создавать свои профили AppArmor/SELinux для более жёсткого контроля доступа контейнера к системе (например, запретить ему обращаться к определённым подсистемам ядра совсем). Эти LSM добавляют слой защиты, хотя требуют опыта для настройки.

8. Безопасность образов на уровне supply chain: Следите, откуда берутся образы. Используйте официальные образы или образы из доверенных источников. Не скачивайте случайные `username/imagename` с Docker Hub без проверки. Включите **Content Trust (Docker Notary)** для верификации подписи образов – Docker умеет подписывать образы (команда `docker trust sign`) и затем при включении DOCKER_CONTENT_TRUST=1 будет загружать только подписанные издателем образы ⁵⁶ . Это предотвращает подмену образов в регистре. Также генерируйте и публикуйте SBOM (Software Bill of Materials) для ваших образов – чтобы точно знать, что внутри (инструменты Syft, Docker sbom). Это помогает отслеживать уязвимые компоненты.

9. Безопасность CI/CD для Docker: Runner-ы, выполняющие `docker build`, тоже требуют защиты. Если у вас shared-раннеры, они должны быть изолированы (идеально – на короткоживущих VMs), т.к. сборка образа – потенциальный вектор (Dockerfile может запускать произвольный код). Следите, чтобы в pipeline не было утечек секретов (например, не передавайте пароли в командной строке Docker, используйте переменные среды и secrets).

10. Контроль доступа: Ограничьте доступ к Docker сокету. Только доверенным пользователям или сервисам должен быть доступен `/var/run/docker.sock`. Помните, что членство в группе `docker = root-доступ` ⁵⁷ ⁵⁸. Не раздавайте его всем. Если нужно предоставить ограниченный доступ, смотрите в сторону Docker REST API + auth проху с фильтрацией команд.

Реальные практики безопасности включают автоматическое сканирование образов на этапе CI (например, ежедневное сканирование базовых образов), применение **CIS Docker Benchmark** – это набор ~100 рекомендаций по конфигурированию Docker (существует скрипт Docker Bench Security, который проверяет хост и демон Docker на соответствие лучшим практикам: файервол, права socket, использованные флаги демона и т.п.). Например, CIS Benchmark требует, чтобы демон Docker не слушал TCP без TLS, чтобы по умолчанию профили `seccomp` включались, чтобы был включен `user namespace remapping` и т.д. Выполнение этих рекомендаций снижает риск эксплуатации.

Подводя итог, безопасность Docker – это многослойная задача: от малого размера и обновлённости образов (меньше уязвимостей), до правильно настроенного рантайма (изоляция, минимальные права), до безопасного окружения запуска (актуальный хост, без излишнего доступа). Соблюдение перечисленных мер (не запускать от `root`, сканировать образы, беречь секреты, обновлять и ограничивать) значительно повысит надёжность контейнеризированного приложения на продакшене.

Хранение данных: тома, маунты и tmpfs

Контейнеры по своей природе эфемерны: данные, записанные во внутреннюю файловую систему контейнера, пропадут вместе с контейнером (при его удалении). Docker предлагает несколько способов хранения данных вне контейнера:

- **Тома (Volumes)**
- **Прямые маунты (Bind Mounts)**
- **tmpfs (в памяти)**

Рассмотрим их различия и применение.

Docker Volumes (тома)

Том – это управляемое Docker хранилище, находящееся на хосте, но изолированное от файловой системы хоста (в смысле, не привязано к конкретному пути пользователя). Когда вы создаёте том командой `docker volume create`, Docker размещает его (по умолчанию) в каталоге `/var/lib/docker/volumes/<имя>` ⁵⁹. Тома можно затем монтировать в контейнеры: при запуске `docker run -v имя_тома:/путь/внутри` или через `--mount type=volume`. Данные, записанные контейнером в том, сохраняются независимо от жизненного цикла контейнера.

Почему тома – основной способ работы с данными: - Тома не зависят от структуры директорий хоста, ими управляет Docker, поэтому их легче переносить между хостами и бэкапить ⁶⁰. - Они работают как с Linux, так и с Windows-контейнерами, одинаково через Docker CLI ⁶¹. - Возможно совместное использование тома несколькими контейнерами (например, два контейнера могут подключить один именованный том и обмениваться через файлы) – при этом Docker следит, чтобы данные тома не удалились, пока хотя бы один контейнер его использует ⁶². - Производительность: запись на том, как правило, быстрее, чем в слое контейнера, т.к. минует копирование через storage driver ⁶³. Запись в том идёт напрямую на хостовой диск, тогда как

запись внутри контейнера должна пройти через union-файловую систему storage-драйвера, что медленнее. - Тома проще бэкапить – достаточно скопировать содержимое каталога тома на хосте или воспользоваться `docker run --volumes-from` для временного контейнера, делающего бэкап (в Docker docs есть примеры бэкапа томов) ⁶⁴ ⁶⁵ .

Docker по умолчанию создает *анонимные тома*, когда в Dockerfile указана инструкция `VOLUME` или при запуске контейнера, если использовать `-v /путь` без имени тома. Такие тома сложно отслеживать, поэтому обычно лучше явно создавать именованные тома.

Когда использовать тома: - Хранение данных БД, чтобы переживать перезапуски контейнеров (как в примере с `db_data` выше). - Разделение данных между контейнерами (например, контейнер приложения и контейнер backup-скрипта могут разделять том с файлами). - Если нужно высокопроизводительное хранилище: тома, как правило, быстрее, чем bind mount. - Мультиплатформенная поддержка: на Windows-хосте тома также работают (они хранятся внутри специального path Docker).

Ограничения: том не привязан к пользователю хоста, и хост-приложения не могут легко (по именам) обращаться к нему. Если нужно прямое взаимодействие с файлами с хоста, то, возможно, нужен bind mount ⁶⁶ .

Тома можно использовать и в Docker Compose: в разделе `volumes` объявляются, а у сервисов монтируются. Compose может автосоздавать тома, если они не созданы заранее.

Bind Mounts (привязанные маунты)

Bind mount – это монтирование конкретной директории или файла хоста в файловую систему контейнера. В отличие от томов, bind mount напрямую зависит от пути на хосте и его содержимого. Задаётся либо флагом `-v /host/path:/container/path`, либо `--mount type=bind,source=/host/path,target=/container/path`.

Применение: - Разработка: монтирование исходного кода с хоста в контейнер для тестирования без перестроения образа. Например, `-v "$PWD":/usr/src/app` – изменения в коде на хосте видны внутри контейнера. - Доступ к конфигурационным файлам хоста или устройствам: можно примонтировать `/etc/hosts` или `/dev/ttyUSB0` и т.п. внутрь контейнера, если нужно. - Легкое извлечение результатов работы контейнера на хост: например, контейнер генерирует отчет в `/out`, примонтированный bind mount позволяет сразу увидеть файл на хосте.

Плюсы и минусы: Bind mount дает полную гибкость – контейнер видит реальную папку. Однако: - **Портативность:** Compose-файл с bind mount будет специфичен для окружения (не у всех разработчиков есть папка `/home/user/data`). Тома в этом смысле портативнее. - **Безопасность:** контейнер при bind mount может потенциально навредить хостовой файловой системе, если дать ему root-права на чувствительный путь. Тома изолированнее – они обычно в `/var/lib/docker` и не дают доступа к остальному. - **Производительность на не-Linux хостах:** на Windows/Mac bind mount выполняется через сетевую шару между VM Docker и хостом, что может значительно замедлять I/O (медленнее, чем том). Рекомендуется по возможности использовать тома или WSL2-расположение в Windows для интенсивного I/O.

Bind mount не управляется Docker – Docker не знает, что внутри, не умеет перечислять или чистить их (в `docker volume ls` они не отображаются). Их жизнь полностью зависит от того, прописаны ли они в работающих контейнерах.

Выбор между томом и bind mount: Docker сам рекомендует: если данные **должны быть доступны как хостом, так и контейнером** (напрямую редактируемые файлы), берите bind mount; если просто нужно постоянное хранилище для контейнера и бэкапы – лучше том ⁶⁷ ⁶⁶. Тома чаще применяются в продакшен-развертываниях (БД, загруженные пользователями файлы и пр.), а bind mounts – в девелопменте и в специфичных интеграциях.

tmpfs (in-memory) том

tmpfs-маунт – это монтирование временной файловой системы в памяти (RAM) внутрь контейнера. Данные на tmpfs не сохраняются на диск ни в контейнере, ни на хосте, и пропадают при остановке контейнера. По сути, это аналог tmpfs в Linux или размещения файлов в /dev/shm.

Docker позволяет при запуске указать --tmpfs /path (простая форма) или --mount type=tmpfs,destination=/path,tmpfs-size=64m. Последнее даёт контроль над размером (в байтах). Tmpfs-монтирование происходит *только на Linux-хостах*; на Windows/macOS оно неприменимо, т.к. требует Linux-ядра.

Когда использовать tmpfs: - Хранение чувствительных данных во время выполнения, которые не должны попасть на диск. Например, ключи шифрования, которые нужны только в памяти. - Кеширование для быстрого доступа: например, если приложение активно пишет во временные файлы, tmpfs ускорит операцию (RAM быстрее диска). - IPC и буферизация – Docker по умолчанию предоставляет /dev/shm (как tmpfs) для POSIX shared memory. Можно увеличить его размер флагом --shm-size при запуске контейнера (по умолчанию /dev/shm = 64Мб). - **Важно:** tmpfs учитывает ограничение по памяти контейнера. Если контейнер ограничен 512Мб RAM, tmpfs тоже съедает из этого лимита.

Использование: `docker run --tmpfs /app/cache:rw,size=100m myimage` – создаст внутри контейнера каталог /app/cache в памяти с размером 100Мб. В Docker Compose tmpfs можно задать так:

```
services:
  app:
    tmpfs:
      - /app/cache:size=100m
```

Плюсы: очень быстрый I/O (без обращения к диску), никакого сохранения на диск (безопасность).

Минусы: данные могут привести к росту потребления RAM, потеряны после перезапуска.

Docker сам рекомендует: если данные короткоживущие и не нужны вне контейнера – tmpfs mount может повысить производительность и не изнашивать диск ⁶⁸.

Сводная таблица сравнения

- **Volumes (тома):** управляются Docker, хранятся в специальном месте. **Использование:** постоянные данные, шаринг между контейнерами, продакшен. **Про:** удобство (CLI-управление ⁶⁹, бэкап), скорость, независимость от хоста. **Минус:** не доступен напрямую хосту по имени (но можно найти путь), дополнительный уровень абстракции.

- **Bind mounts:** привязаны к конкретному пути хоста. **Использование:** доступ к данным хоста, dev-среда. **Про:** прозрачен – контейнер видит именно файлы хоста, можно легко читать/писать снаружи контейнера. **Минус:** зависит от ОС/пути (менее переносим), возможны проблемы прав (UID/GID внутри контейнера vs на хосте), на Windows/Mac – медленнее.
- **Tmpfs:** данные в памяти. **Использование:** временные данные, секреты на runtime. **Про:** быстрый, не оставляет следов. **Минус:** ограничен RAM, не сохраняет данные, работает только на Linux.

В production-развёртываниях часто используются named volumes для всего, что требует сохранности, а bind mounts – только для случаев, где нужно явно дать контейнеру что-то с хоста (например, Unix-сокеты приложения, SSL-сертификаты из host-файла, и т.п.). Tmpfs примечателен в задачах, где запись на диск нежелательна (кэш, сессии, временные расчёты).

Знание разных типов хранилища Docker помогает спроектировать контейнерное приложение правильно: удерживать важные данные вне контейнеров, но с нужным уровнем изоляции и производительности.

Логирование, мониторинг и алертинг в контейнеризированной среде

При переходе к контейнерам меняется подход к логам и мониторингу: вместо лог-файлов на конкретных серверах мы оперируем потоками stdout/stderr и метриками множества динамических контейнеров. Рассмотрим, как собирать логи, мониторить контейнеры и настраивать оповещения.

Логирование контейнеров

Логи приложений в Docker обычно пишутся в stdout/stderr процесса. Docker по умолчанию перехватывает эти потоки и сохраняет их через драйвер логирования *json-file*: для каждого контейнера создаётся файл в `/var/lib/docker/containers/<id>/<id>-json.log`, куда пишется JSON с записями логов. Затем команды `docker logs` читает оттуда.

Проблема: по умолчанию эти логи растут без ограничения. В продакшне рекомендуется включить ротацию логов для драйвера json-file (например, в конфиге демона `/etc/docker/daemon.json`):

```
{
  "log-driver": "json-file",
  "log-opts": {
    "max-size": "100m",
    "max-file": "3"
  }
}
```

Это ограничит размер логов контейнера до ~300Мб (3 файла по 100Мб) и предотвратит заполнение диска.

Альтернативно, Docker предоставляет **различные драйверы логирования** ⁷⁰: - `syslog`: отправляет логи контейнера в системный syslog демона (что дальше можно перенаправить на удалённый syslog-сервер). - `journald`: интеграция с системным journald на Linux. - `fluentd`: напрямую шлёт логи в Fluentd (должен быть запущен агент fluentd на хосте). - `gelf`: отправляет на Graylog. - `awslogs`, `splunk`, `gcplogs` и др. – драйверы для облачных лог-сервисов.

Выбор драйвера зависит от инфраструктуры. Например, в Kubernetes принята схема: каждый контейнер пишет JSON-логи, а *sidecar* или *daemonset* (fluent-bit, fluentd) собирает их и передаёт в ElasticSearch/CloudWatch/специфичные хранилища. В Docker Swarm можно тоже настроить каждый контейнер сразу на облачный лог (но тогда потеряете локальные копии).

Если не хотите менять драйвер, популярный подход: - Запускать агент, который читает файлы json-log и отправляет централизованно. Например, **Filebeat** (Elastic) или **Fluent Bit**, настроенные на `/var/lib/docker/containers/*/*-json.log`. - Либо использовать **docker logs --follow** через API – но это неудобно для массового сбора.

Проще всего – настроить Docker на syslog или fluentd и запустить syslog/fluentd-агент на хосте. Fluentd особенно распространён: Docker имеет встроенный fluentd-драйвер ⁷¹, а Fluentd может собирать и перенаправлять дальше (например, в Elasticsearch – классический стек EFK).

Логирование в Compose: В docker-compose.yml можно задать для каждого сервиса драйвер и опции:

```
logging:
  driver: "json-file"
  options:
    max-size: "10m"
    max-file: "5"
```

Или `driver: syslog` и т.д. Это позволяет централизованно задать политику логов.

Практика: В продакшене часто организуют **централизованное логирование**: все контейнеры шлют логи в единое хранилище, где их можно фильтровать и искать. Это может быть ELK/EFK (Elasticsearch+Kibana), Graylog, Splunk, CloudWatch, Loki+Grafana. Например, Loki (от Grafana) вместе с Promtail (агент на хосте) собирает логи с минимальными ресурсами и хранит их с индексами по меткам контейнеров.

Совет: структурируйте логи приложения (например, в JSON), тогда передача их через Docker json-file не искажает структуру. Либо переходите на fluentd-driver, который может передавать любую строку как есть.

Logrotate: Если не хотим менять драйвер, можно просто настроить системный logrotate для файлов Docker. Но встроенный `max-size` опции делают то же, так что лучше воспользоваться ими.

Мониторинг контейнеров

Мониторинг включает сбор метрик: использование CPU, памяти, сети, диска, а также метрик самого приложения.

Docker предоставляет команду `docker stats` для просмотра live-метрик по контейнерам (CPU%, память, I/O). Но для мониторинга всего кластера нужен систематический подход: - **cAdvisor (Container Advisor)**: Open-source инструмент от Google, отслеживает в реальном времени метрики использования ресурсов *каждого контейнера*. Его можно запускать как контейнер на каждом хосте:

```
docker run -d --name=cadvisor -p 8080:8080 \
-v /var/run/docker.sock:/var/run/docker.sock \
-v /sys:/sys:ro -v /var/lib/docker/containers:/var/lib/docker:ro \
google/cadvisor:latest
```

cAdvisor собирает метрики и предоставляет их по HTTP в формате Prometheus. Он показывает CPU/Mem/Network/Filesystem usage per container, а также агрегирует по image, по поду (в Kubernetes). С Grafana можно подключить эти метрики для визуализации (есть готовые дашборды Grafana для cAdvisor) ⁷².

- **Docker Engine metrics endpoint:** Docker daemon сам может экспонировать метрики (с версии ~20) напрямую для Prometheus ⁷³. В конфиге запуска dockerd можно указать `--metrics-addr 0.0.0.0:9323 --metrics-interval=15s` - и тогда на порту 9323 будут метрики по докеру. Но это больше про состояние самого демона и суммарно контейнеров; cAdvisor обычно даёт более подробную инфу по каждому контейнеру ⁷⁴.
- **Сторонние агенты:** Такие как Datadog agent, New Relic, Zabbix-docker etc., которые устанавливаются на хосте и сами дергают Docker API для данных. Datadog, например, автодетектит контейнеры, собирает метрики и логи, рисует дашборды.
- **Application metrics:** Не забывайте о самих приложениях - внутри контейнеров приложения могут экспонировать метрики (например, Prometheus endpoint, StatsD и прочее). В контейнерном окружении хорошо иметь sidecar или экспортеры. Но если говорим только о Docker, то сбор метрик приложений аналогичен как на обычных серверах: либо приложение шлет метрики во внешнюю систему (push), либо система опрашивает (pull) через сети Docker (например, Prometheus сервер может находиться в другом контейнере/хосте и опрашивать метрики по имени контейнера при общей сети).

Алертинг (оповещения): Налаживается поверх собранных метрик и логов: - Для метрик: если используете Prometheus, то **Alertmanager** сможет сигнализировать (email, Slack, PagerDuty) при достижении условий: например, контейнер перезапускается чаще N раз в час, память >90% лимита, CPU недоступен и т.д. Prometheus можно настроить на правила (recording rules) для расчёта, например, *ratio памяти контейнера к лимиту*. - Для логов: если логи централизованы в Elastic/Graylog/Splunk, там можно задать поиск по шаблону (например, ERROR или stack trace) и настроить оповещение при появлении новых записей. Инструменты вроде ELK Stack имеют Alerting (платный в Elastic, либо через watcher), Graylog - Alerts в open-source, Splunk - тоже. Более современный - Grafana Loki с Alertmanager через графит-синтаксис.

- **Docker event alerts:** Docker daemon генерирует *события* (docker events) при действиях: старт/стоп контейнера, потеря healthcheck. В теории, можно слушать `docker events` и реагировать (например, скриптом слать сообщение, если контейнер упал). Однако в масштабах продакшена лучше использовать вышеописанные системы мониторинга, т.к. они надежнее.

Практики мониторинга: В продакшене часто деплоят стек: *Prometheus + Grafana* для метрик, *EFK* (*Elasticsearch-Fluentd-Kibana*) или *Loki + Grafana* для логов, и интегрируют Alertmanager. Например, команда DevOps настраивает дашборды: общее число контейнеров, использование CPU памяти по сервисам, графики рестартов. Alerts: “меньше 1 контейнера сервиса X работает” (значит, он упал), “память контейнера > 95% лимита 5 минут” (намёк на утечку), “в логах контейнера появилась фраза Exception”.

Отдельно стоит упомянуть **Healthcheck** Docker-контейнеров: если вы определили HEALTHCHECK, Docker маркирует контейнер healthy/unhealthy. В режиме Swarm orchestrator может перезапускать не healthy. В standalone Docker можно настроить чтобы unhealthy контейнер автоматически перезапускался (через внешние скрипты или ручную мониторинг). Или хотя бы `docker ps` покажет статус (healthy/unhealthy) в колонке STATUS. Это быстрый индикатор для админа, но для алертинга лучше задействовать полноценный мониторинг.

Логирование и мониторинг в Compose/Swarm: Если у вас несколько хостов и Docker Swarm, обратите внимание на встроенные инструменты: Docker предоставляет *Swarm mode* с сервисами, где можно внедрить агент на каждый узел (global сервис), и использовать overlay networks для сбора. Например, запустить глобально cAdvisor и один Prometheus, опрашивающий cAdvisor на каждой ноде.

Вне Swarm – используйте Docker Compose, чтобы поднять себе monitoring stack локально: есть готовые примеры *cadvisor+prometheus+grafana* в `docker-compose.yml` ⁷⁵, или *EFK stack* на одном хосте.

Пример: сбор логов через Fluentd

Запустим Fluentd как лог-агент на хосте с Docker. В конфиге Docker Engine (`daemon.json`) укажем:

```
{
  "log-driver": "fluentd",
  "log-opts": {
    "fluentd-address": "localhost:24224",
    "tag": "{{.ImageName}}/{{.Name}}"
  }
}
```

Это заставит все контейнеры отправлять логи локально на Fluentd. Fluentd, запущенный на 24224, примет их и дальше по своему конфигу, например, отправит в Elasticsearch. Поле `tag` мы задали шаблоном: он подставит имя образа и имя контейнера ⁷⁶, чтобы мы видели источник лога. Такой подход разгружает Docker host от хранения логов и сразу поставяет их в централизованное хранилище.

Пример: мониторинг с Prometheus

Допустим, у нас 10 хостов с Docker. Мы можем запустить один контейнер Prometheus, сконфигурировав ему statically 10 адресов (метрика Docker или cAdvisor на каждом). Или на каждой ноде – промеевский экспортер Node Exporter + cAdvisor, а Prometheus опрашивает. Grafana затем подключается к Prometheus и отображает метрики.

С помощью меток (labels) Prometheus знает, какие контейнеры на каком хосте, к какому сервису принадлежат (можно пробрасывать переменные окружения как метки, или использовать имена контейнеров по шаблону). Например, метка `container_label_com_docker_compose_service="web"` может указывать сервис Compose.

Оповещения: Alertmanager (входит в состав Prometheus stack) настроим, например, на правило: если контейнер сервис "web" меньше 1 экземпляра в течение >1 минуты – шлём уведомление "Web service down".

В заключение: настройте протоколирование (logging) так, чтобы логи не терялись при удалении контейнера и были доступны для анализа – это критично для отладки. И мониторьте и сам Docker (демон, дисковое пространство для образов/томов) – например, `docker system df` можно периодически проверять или метрику free space на `/var/lib/docker`. В высоконагруженной среде важно отслеживать, чтобы не переполнялись тома с логами или образами.

Инструменты такие как **Portainer** (веб-интерфейс для Docker) тоже предоставляют базовые графики и логи, но для масштабного продакшена всё же нужны специализированные решения.

BuildKit, Registry и продвинутое сетевое взаимодействие

В этом разделе – о современных возможностях сборки с BuildKit, работе с регистраторами образов и тонкой настройке сетей Docker.

BuildKit и Docker Buildx

BuildKit – это новый бэкенд сборки Docker, разработанный для повышения производительности и возможностей. Он включён по умолчанию в Docker Desktop и Docker Engine (начиная с Docker 19.03, активируется переменной окружения или флагом). BuildKit привносит:

- **Параллельную сборку слоёв:** традиционный `docker build` шёл последовательно, BuildKit может выполнять независимые шаги параллельно, что ускоряет сборку на многоядерных системах.
- **Кэширование более гибкое:** BuildKit умеет кэшировать результаты RUN даже при изменениях (partial caching), умеет загружать/выгружать кеш между сборками. С помощью `--cache-from` и `--cache-to` можно сохранять кэш в registry или локальный файл, ускоряя CI-сборки.
- **Секреты и SSH-агент:** BuildKit позволяет безопасно передавать секреты. Как упоминалось, `RUN --mount=type=secret` – без BuildKit недоступно. Также `--mount=type=ssh` позволяет прокидывать SSH-ключ для, например, приватного `git clone` в Dockerfile.
- **Многоступенчатая сборка условно:** BuildKit может пропускать неиспользуемые build stage (если артефакты из них не нужны), экономя время.
- **Frontend Dockerfile синтаксис 1.x:** BuildKit развивает синтаксис Dockerfile – например, инструкции `#syntax=docker/dockerfile:1.4` позволяют использовать новые функции: `RUN --mount=type=cache` (для кэширования директорий между шагами, например, кеш .npm или apt), `COPY --link` (для более эффективной копии), `RUN --mount=type=bind` (чтение файлов без включения в контекст).

Чтобы убедиться, что BuildKit используется, можно выставить `DOCKER_BUILDKIT=1`. Также Docker CLI имеет команду `docker buildx` – это интерфейс над BuildKit builder'ами. `docker buildx build` позволяет: - Собирать образы для нескольких платформ одновременно (multi-

arch) с помощью QEMU эмуляции (либо using buildx + buildkit on different nodes). Например, `docker buildx build --platform linux/amd64,linux/arm64 -t myapp:latest .` за один проход создаст манифест обобщённого образа. - Прямо пушить результат сборки в registry (`--push`), минуя хранение локально. - Использовать контейнеризованные builder'ы и контролировать их (команды `docker buildx create/use/inspect`). - Кэш. В buildx можно настроить remote cache backend (например, Amazon S3 или Docker Registry с помощью `type=registry`), чтобы даже между разными CI сборками разделять кэш слоёв.

Пример: в GitHub Actions мы использовали `docker/setup-buildx-action` – он создает builder. Затем `docker/build-push-action` под капотом вызвал `docker buildx build --push ...`. BuildKit параллельно установил зависимости, собрал образы, и сразу отправил их, не держа лишний слой локально.

BuildKit можно использовать и локально для ускорения. Например, сборка с `--mount=type=cache,target=/root/.cache/pip` ускорит повторный `pip install` между сборками.

Совет: Всегда используйте BuildKit, если нет специфических причин обратной совместимости. Он стабилен и постоянно развивается, тогда как классический builder устарел. К тому же, некоторые новые фичи Dockerfile (например, `COPY --chmod=0755` или поддержка heredoc в RUN) доступны только с BuildKit.

Регистры (Registry) и управление образами

Docker Registry – хранилище образов. Самый известный – Docker Hub (hub.docker.com), но в компании часто используют приватные регистры: - **Docker Hub (публичный/приватный):** Ограничения на приватные репо в бесплатном плане (несколько, с лимитами на количество pull). - **GitLab Container Registry:** в составе GitLab, URL обычно `registry.gitlab.com/namespace/project/image`. - **GitHub Container Registry (GHCR):** регистр от GitHub (ghcr.io) или прежний Docker pkg.github. - **Облачные:** Amazon ECR, Google Container Registry (GCR) / Artifact Registry, Azure Container Registry (ACR) – интегрируются с облаками. - **Свой Registry (Harbor, JFrog Artifactory, Quay, или open-source distribution):** Можно поднять контейнер `registry:2`, который является официальной реализацией Docker Registry. Он достаточно простой, поддерживает хранение на локальном диске или в S3/кластерах.

Практики именования и тегирования: Рекомендуется тегировать образы семантически: - `:<версия>` – например, `myapp:1.4.2` для версии релиза. - Дополнительно `:commit` или `:buildnumber` для уникальности CI сборки. - Избегать длительного использования `:latest` в продакшене – latest движется и может внезапно обновиться при pull, лучше явно указывать версии. Latest хорошо только для development, да и то – не всегда. Если используете latest, скорее всего CI должен явно перетегировать новую latest. - Если образ multi-arch, Docker Registry хранит manifest list – по тегу, а у него внутри платформы. Buildx сам публикует manifest list. Убедитесь, что pull с нужной платформы тянет правильное (например, `docker pull myimage:tag` на arm64 стянет arm64 вариант, если multi-arch сделан). - Если храните много образов, внедрите политику ретенции: чистить очень старые (например, оставить только образы за последний год или 50 шт). Docker Registry API и многие UI (Harbor) поддерживают очистку. - Sign & verify: продумайте, нужно ли вам подписывать образы (Docker Content Trust) – для критичных образов в production это повышает уверенность, что образ не подменён.

Docker Registry authentication: Docker CLI использует `docker login` (Basic auth) для частных регистров. В CI секреты (логин/пароль или токен) должны храниться безопасно (в GitLab CI – Variables Masked/Protected, в GitHub – Actions Secrets). В Kubernetes, если вы деплоите, нужны `Secrets type=docker-registry`.

Свой сервер Registry: Официальный образ `registry:2` может быть запущен так:

```
docker run -d -p 5000:5000 --restart=always --name registry \
-v /opt/registry/data:/var/lib/registry \
registry:2
```

Теперь push к `localhost:5000/myimage:tag` сохранит образ в `/opt/registry/data`. Можно защитить его Basic Auth (через реверс-прокси nginx) или использовать auth server. Многие вместо этого идут к готовым решениям, как Harbor – он предоставляет webUI, LDAP, сканирование Clair/Trivy, репликацию.

Оптимизация доставки образов: В продакшене часто образы тяжёлые, и есть механизмы ускорения: - Локальные Registry mirror – в конфиге Docker Engine можно прописать зеркала для Docker Hub (чтобы частые pull'ы происходили из локального кеша). Например, Nexus может выступать как прокси-репозиторий. - Content Delivery Network (CDN) – Docker Hub Enterprise или облачные registry размещают кэш ближе к серверам. Например, GCR/AR автоматически хранят образы в регионах, ACR – тоже. - Deduplication: Docker pull вытягивает только слои, которых нет. Убедитесь, что используете общие базовые образы, чтобы на узле они уже были. Например, если у вас 10 микросервисов на `python:3.11-alpine`, то базовый слой будет переиспользован.

Registry и CI/CD: После сборки CI пушит в registry, далее деплой-система (будь то Ansible, Kubernetes или Swarm) тянет оттуда. Хорошей практикой является *immutable image tags* – не перезаписывать один и тот же тег новым содержимым, а использовать новый (или контент-адресный, как git SHA). Тогда вы точно знаете, что версия деплоя соответствует конкретному образу. Можно использовать дополнительный тег `latest` для "последней версии", но не полагаться на него для отката.

Безопасность Registry: Ограничьте доступ – приватные registry требуют аутентификации. Ограничьте кто может пушить (чтобы никто не смог запустить малварь под именем вашего сервиса). Внедрите сканирование – Harbor, GitLab, GitHub CR имеют встроенный сканер CVE (часто на базе Trivy/Clair). Включите подписывание (Notary v1 в Docker Hub, или в Harbor можно включить Cosign signatures). Supply chain security – большая тема, но registry – ключевой узел (не дайте скомпрометировать его).

Продвинутое сетевое взаимодействие Docker

Docker предоставляет несколько сетевых драйверов для разных сценариев:

- **Bridge network (мост)** – стандарт для контейнеров на одном хосте. По умолчанию у Docker есть сеть `bridge` (docker0 интерфейс на хосте). Контейнеры, запущенные без `--network`, попадают туда, но **не видят друг друга по именам** (legacy link-only). Лучше создавать пользовательские bridge-сети: `docker network create mynet`. В них:
 - Контейнеры получают IP из приватной подсети (Docker выбирает, например, 172.18.0.0/16).

- Docker запускает DNS-сервер, привязанный к этой сети, и каждый контейнер может по имени обращаться к другим.
- Можно подключать контейнеры динамически.
- Изоляция: контейнеры в разных bridge сетях не видят трафик друг друга (разные виртуальные свитчи).
- Bridge сети поддерживают iptables NAT для выхода в интернет и порт-форвардинг для входа (это Docker делает автоматически для published ports).
- **Host network** – контейнер разделяет сеть с хостом. Контейнер не получает отдельного IP, а использует хостовой. Это означает:
 - Порты, которые слушает приложение в контейнере, становятся портами хоста. Нет изоляции портов (два контейнера на host-сети не могут слушать один порт).
 - Производительность максимальная (нет оверхеда NAT).
 - Полезно для сетевых сервисов, где нужен полный доступ к хостовой сети (например, снифферы, или если нужно объявлять сервис по mDNS, etc).
 - На Windows host network ограничена, а на Mac (Docker Desktop) – host сеть работает только для Linux-контейнеров внутри VM, но не привязана к внешней Mac (Docker Desktop делает специальные прокси).
- Используется: например, `docker run --network host nginx` запустит nginx, доступный сразу на 80 порту хоста.
- **None network** – полностью отключенная сеть. Контейнер запускается без сетевого стека (только lo интерфейс). Используется для изолированных задач, не требующих сети.
- **Macvlan** – спецрежим: Docker может "прикрепить" контейнер к физической сети хоста, дав ему отдельный MAC-адрес и IP-адрес в той же сети, что и хост. Контейнер становится как бы полноправным узлом L2 сети. Это удобно, если надо, чтобы контейнер был доступен в локальной сети наравне с хостами (без NAT). Пример: запуск контейнеров, имитирующих физические устройства в сети. Минусы: macvlan сложно комбинировать с обычным доступом (хост не может напрямую общаться с своими macvlan-контейнерами без настроек), и требует специфичной поддержки (не работает на WiFi-интерфейсах в режиме infra из-за ограничений). Но в целом, Macvlan – мощный инструмент для legacy-сетей.
- **Overlay network** – распределенная сеть для Docker Swarm (или Docker Enterprise, Kubernetes via Libnetwork). Она позволяет контейнерам на разных хостах находиться в одной виртуальной L2 сети поверх VXLAN. В режиме Docker Swarm достаточно `docker network create --driver overlay myoverlay` и все worker-ы подключатся. Контейнеры сервисов (docker service) видят друг друга по имени. Overlay обеспечивает шифрование (опционально) между хостами и автоматическое управление членством. Для Swarm это основа многохостовой коммуникации. Без включения Swarm overlay работать не будет (в обычном Docker Engine overlay не создается, нужна активная Swarm-конфигурация даже если одиночный узел). Если использовать Kubernetes, там свои механизмы (CNI плагины), но Docker's overlay в Swarm – аналог flannel/calico и т.д.

Дополнительные моменты:

- **DNS в Docker:** Внутри контейнера Docker по умолчанию настроит `/etc/resolv.conf` указывать на Docker встроенный DNS (на базе Embedded DNS server). Этот DNS отвечает на

запросы имен контейнеров на той же сети. Если имя не найдено, он форвардит на внешние DNS (обычно от хоста). Можно вручную указать `--dns` сервер или `--dns-search` домен, если нужно.

- **Port publishing:** Флаг `-p` (или `ports:` в Compose) делает DNAT с хоста на контейнер. Docker может публиковать порт на всех интерфейсах хоста (0.0.0.0) или конкретном (`-p 127.0.0.1:8000:80` – только localhost). Учтите, что без `-p` сервис не доступен извне, но доступен для других контейнеров в той же сети.
- **Firewall и Docker:** Docker на Linux манипулирует iptables: создаёт цепочки DOCKER для NAT и FILTER. Если вы вручную управляете fw, нужно учитывать правила Docker или использовать флаг `--iptables=false` (и тогда самому настроить маршрутизацию). Для корпоративных окружений: помнить про **Packet filtering** – Docker создаёт интерфейс docker0 (bridge), который может конфликтовать с iptables-policies. Docker docs рекомендует открыть определенные порты для overlay networks (VXLAN порт 4789/UDP, etc) если multi-host.
- **Продвинутый сетевой мониторинг:** Можно смотреть трафик внутри контейнеров с помощью `docker exec` (tcpdump, iftop если установить в контейнер), либо на хосте `tcpdump -i docker0`. Docker не предоставляет встроенных средств мониторинга сети, но cAdvisor/Prometheus Node Exporter могут давать throughput stats.
- **Прокси и Docker:** Если хост стоит за корпоративным прокси, надо настроить daemon (systemd drop-in) чтобы Docker тянул образы через прокси, и контейнерам передавать `http_proxy`. Это не про сеть Docker, но часто сталкиваются.

Практические кейсы: - Имеем несколько контейнеров, которым не нужно общаться друг с другом – поместите их в разные bridge сети, для безопасности (правило: "каждое приложение – своя сеть"). - Нужно чтобы два микросервиса общались – одна сеть, и использовать hostnames. Например, `db` и `app` на сети backend_net: тогда в конфиге app можно указать `DB_HOST=db`. - Хотим отказоустойчивость: 2 инстанса app на разных хостах, и они должны видеть сервис БД – тут надо или все на одном хосте (bridge), или задействовать Swarm overlay / Kubernetes. - Low-level: Docker позволяет опцию `--network-alias` при запуске – выдать контейнеру дополнительные DNS-алиасы в сети. Полезно, если один контейнер должен отзываться по нескольким именам. - Можно подключать контейнер к нескольким сетям (каждый получит по IP в каждой). Например, контейнер-связующее звено между двумя сетями. - **Legacy links:** Старый способ `--link container:alias` – устарел, use networks+DNS instead. Links имели ограничения и сейчас практически заменены сетями. - **IPv6:** Docker поддерживает dual-stack. Нужно включить IPv6 в демоне, задать IPv6-подсеть. Тогда контейнеры могут получать IPv6 адреса тоже.

Network security: Docker networks – удобство, но помните: на одном хосте контейнеры разных пользователей могут случайно оказаться в default bridge и тогда между ними нет изоляции (они могут общаться по IP). Поэтому в мульти-арендном Docker лучше явно сажать контейнеры разных клиентов на разные пользовательские сети и не смешивать. Или вообще использовать network none, кроме случаев явного разрешения. Firewall/iptables можно запрещать трафик между определенными network (Docker out of box не делает меж-сетевые фильтры).

Performance tuning: Для высоконагруженных сетевых сервисов, NAT Docker может стать узким местом. Переход на macvlan или host network избавляет от overhead NAT – может повысить пропускную способность и снизить latency. Но host/macvlan отнимают у вас гибкость (нет

изоляции портов). Решение – разносить, например, front (LB) на host-network, а backend – на bridge. Или использовать NodePort в orchestrator.

В целом, Docker сети позволяют моделировать практически любую архитектуру: изолированные подсети, общие сети, прямое подключение к LAN, etc. **Advanced tip:** Можно вручную присоединять контейнер к существующему интерфейсу хоста с `--network container:...` или используя `bridge/ipvlan` driver, но это выходит за рамки стандартных сценариев.

Шпаргалка Docker-команд и типовых паттернов

Ниже приведён краткий перечень часто используемых команд Docker и шаблонных приёмов для быстрого ознакомления и напоминания:

• Образы:

- `docker build -t <имя:тег> .` – собрать образ из Dockerfile в текущей директории.
 - `docker pull <образ:тег>` – загрузить образ из Docker Hub или другого registry.
 - `docker push <образ:тег>` – отправить образ в репозиторий (после `docker login`).
 - `docker images` – список локальных образов (аналог `docker image ls`).
 - `docker rmi <образ>` – удалить образ. Флаг `-f` – форсированно (если даже используется).
- **Pattern:** Обновление базового образа: чтобы обновить все образы Alpine до последней версии, можно выполнить:

```
docker pull alpine:latest && docker build --no-cache -t myapp:latest .
```

(Но лучше явно указывать версии вместо latest, и обновлять их вручную.)

• Контейнеры:

- `docker run [опции] <образ>` – создать и запустить новый контейнер. Часто используемые шаблоны:
 - Запуск с интерактивной консолью: `docker run -it --rm ubuntu:latest bash` (запустит `bash` в контейнере `Ubuntu` и удалит контейнер по выходу).
 - Демонизированный сервис: `docker run -d --name nginx -p 80:80 nginx:alpine`.
 - Маунт папки: `docker run -d -v $(pwd)/data:/app/data busybox ls /app/data` (пример запуска `busybox` для просмотра файлов хоста).
- `docker stop <имя|id>` – корректно остановить (SIGTERM + таймаут + SIGKILL).
- `docker rm <имя|id>` – удалить контейнер. Можно комбинировать: `docker rm -f $(docker ps -aq)` чтобы зачистить все контейнеры (осторожно на проде!).
- `docker ps -a` – список всех контейнеров (включая завершённые).
- `docker exec -it <контейнер> sh` – войти в запущенный контейнер (если там есть shell).
- `docker logs -f <контейнер>` – следить за логами (Ctrl+C чтобы прекратить чтение).
- **Pattern:** Копирование файлов: быстро извлечь файл: `docker cp контейнер:/path/to/file ./localdir`. Или наоборот, загрузить файл внутрь: `docker cp my.conf web:/etc/nginx/conf.d/`.

- **Pattern:** *Перезапуск при отладке:* если контейнер падает, можно запустить его временно с override командой, напр: `docker run --entrypoint sh myapp:latest` чтобы попасть в оболочку без запуска приложения.

• Volumes & Storage:

- `docker volume create data_vol` – создать том.
- `docker run -v data_vol:/data ...` – подключить именованный том.
- `docker run -v /host/dir:/data ...` – bind mount директории.
- `docker volume ls` / `docker volume inspect` – список и детали томов.
- `docker volume prune` – удалить неиспользуемые тома.

- **Pattern:** *Бэкап тома:*

```
docker run --rm -v data_vol:/volume -v $(pwd):/backup alpine \
tar czf /backup/data_vol.tar.gz /volume
```

(Создаст архив данных тома через временный контейнер Alpine).

- **Pattern:** *Перенос данных между контейнерами:* запуск контейнера, использующего `--volumes-from`:

```
docker run --rm --volumes-from old_container -v $(pwd):/backup busybox
tar czf /backup/out.tar /data
```

Это позволит достать данные из томов, смонтированных в `old_container`, без прямого доступа.

• Network:

- `docker network create mynet` – создать сеть.
- `docker run --network mynet --name c1 alpine ping c2` – запустить контейнер `c1` и пинговать контейнер `c2` в той же сети.
- `docker network connect mynet container` – подключить существующий контейнер к сети.
- `docker network ls` / `inspect` – управление сетями.
- **Pattern:** *Изоляция сервисов:* создать две сети, `frontnet` и `backnet`. Подключить веб-сервер к обеим (принимает трафик извне на `frontnet` и общается с БД по `backnet`), а базу данных – только к `backnet`. Тогда БД не доступна напрямую извне.
- **Pattern:** *Быстрый доступ к хосту:* использовать special DNS имя `host.docker.internal` (Docker Desktop) или настроить `--add-host`. Например, контейнеру нужно достучаться до сервиса, работающего на хосте – на Windows/Mac можно использовать `host.docker.internal`, на Linux можно передать IP хоста или поднять `macvlan`.

• Docker Compose:

- `docker-compose up -d` – поднять все сервисы по описанию.
- `docker-compose down` – остановить и удалить контейнеры (добавьте `-v`, чтобы удалить созданные тома, если нужно).

- `docker-compose logs -f` – объединённые логи всех сервисов (с префиксами).
- `docker-compose exec <service> <cmd>` – как `docker exec`, но по имени сервиса.
- **Pattern:** *Перезапуск одного сервиса:* `docker-compose up -d web` перезапустит только сервис `web` (если конфиг изменился – перечитает).
- **Pattern:** *.env для Compose:* вынести переменные (например, `DB_PASSWORD`) в `.env`, чтобы не хардкодить их в `yaml`. Потом `docker-compose config` покажет полученную конфигурацию с подставленными значениями (хорошо для отладки).

• System Management:

- `docker info` – общая информация о Docker (версии, кол-во контейнеров, драйверы).
- `docker system df` – "disk free" по Docker объектам: сколько места занимают образы, тома, контейнеры.
- `docker system prune -af` – удаляет всё неиспользуемое без подтверждения (осторожно).
- **Pattern:** *Очистка dangling images:* `docker image prune` удалит только "висячие" слои (не помеченные теги). Больше экономии даст `docker system prune`.
- **Pattern:** *Обновление Docker:* на Linux использовать пакетный менеджер (`apt-get upgrade docker-ce`), на Docker Desktop – через интерфейс/авто-обновление.

• Безопасность и прочее:

- `docker scan <image>` – если у вас Docker от Docker Desktop, есть встроенная команда сканирования (на базе Snyk) для образов, ищущая уязвимости.
- `docker stats` – быстро посмотреть нагрузку контейнеров.
- `docker inspect --format='{{json .Mounts}}' container` – вывести информацию, например, о смонтированных томах в читаемом виде (используя шаблоны Go).
- `docker pull --platform linux/arm64/v8 nginx` – вытянуть образ для другой архитектуры (если поддерживается multi-arch manifest).
- `docker context` – команда для управления контекстами (подключение к удалённым Docker hosts или Docker Desktop contexts like Kubernetes).
- **Pattern:** *Удалённый доступ:* `docker -H tcp://host:2375 info` – подключиться к удалённому демону (если он открыт). Или настроить TLS и подключаться по `tcp://host:2376` с сертом. Docker context позволяет сохранить эти настройки.

Наконец, при возникновении проблем: - Используйте `docker logs` и `docker inspect` щедро – они дадут понимание, что произошло. - `docker events` – увидеть поток событий, полезно чтобы отследить, не перезапускается ли контейнер и почему. - `dmesg` на хосте – если контейнер убит по OOM (out-of-memory), система сообщит в `dmesg` (и `docker inspect` покажет `OOMKilled=true`).

Этот гайд охватывает множество аспектов Docker – от установки до сложных сценариев. Применяя эти знания на практике, вы сможете эффективно строить контейнеризированные приложения, поддерживать их безопасными и надежными в продакшне. **Docker продолжает развиваться**, поэтому следите за новыми возможностями (например, Docker Scout, обновления BuildKit, новые драйверы) и сообществом, чтобы всегда использовать лучшие инструменты и подходы.

1 2 3 4 57 58 **Post-installation steps | Docker Docs**

<https://docs.docker.com/engine/install/linux-postinstall/>

5 **Install Docker on Windows (WSL) without Docker Desktop**

<https://dev.to/bowmanjd/install-docker-on-windows-wsl-without-docker-desktop-34m9/comments>

6 **Issues with Docker Desktop and WSL 2 Integration**

<https://forums.docker.com/t/issues-with-docker-desktop-and-wsl-2-integration/141865>

7 **Install | Docker Docs**

<https://docs.docker.com/engine/install/>

8 **Docker Commands Cheat Sheet | Cherry Servers**

<https://www.cherryservers.com/blog/docker-commands-cheat-sheet>

9 11 12 13 14 15 16 17 18 19 20 21 **Продвинутое конфигурирование Docker Compose (перевод) / Хабр**

<https://habr.com/ru/companies/otus/articles/337688/>

10 **Services top-level elements - Docker Docs**

<https://docs.docker.com/reference/compose-file/services/>

22 23 24 25 **Use Docker to build Docker images | GitLab Docs**

https://docs.gitlab.com/ci/docker/using_docker_build/

26 **GitHub Actions and Docker | Docker Docs**

<https://docs.docker.com/guides/gha/>

27 30 31 32 33 36 37 38 41 42 **Top 20 Dockerfile best practices | Sysdig**

<https://sysdig.com/learn-cloud-native/dockerfile-best-practices/>

28 48 49 50 **Scanning Docker Images for Vulnerabilities: Using Trivy for Effective Security Analysis | by Ramkrushna Maheshwar | Medium**

<https://medium.com/@maheshwar.ramkrushna/scanning-docker-images-for-vulnerabilities-using-trivy-for-effective-security-analysis-fa3e2844db22>

29 **Multi-stage builds - Docker Docs**

<https://docs.docker.com/build/building/multi-stage/>

34 35 43 44 46 47 52 53 54 55 **Docker Security - OWASP Cheat Sheet Series**

https://cheatsheetseries.owasp.org/cheatsheets/Docker_Security_Cheat_Sheet.html

39 40 51 76 **GitHub - goodwithtech/dockle: Container Image Linter for Security, Helping build the Best-Practice Docker Image, Easy to start**

<https://github.com/goodwithtech/dockle>

45 **10 Docker Security Best Practices - Snyk**

<https://snyk.io/blog/10-docker-image-security-best-practices/>

56 **Docker Desktop | Docker Docs**

<https://docs.docker.com/desktop/>

59 60 61 62 63 64 65 66 67 68 69 **Volumes | Docker Docs**

<https://docs.docker.com/engine/storage/volumes/>

70 **Docker Logging Guide Part 2: Advanced Concepts & Best Practices**

<https://www.red-gate.com/simple-talk/devops/containers-and-virtualization/docker-logging-guide-part-2-advanced-concepts-best-practices/>

71 **Fluentd logging driver - Docker Docs**

<https://docs.docker.com/engine/logging/drivers/fluentd/>

72 Monitoring Docker container metrics using cAdvisor - Prometheus

<https://prometheus.io/docs/guides/cadvisor/>

73 Collect Docker metrics with Prometheus | Docker Docs

<https://docs.docker.com/engine/daemon/prometheus/>

74 Standalone Docker monitoring native metrics vs cAdvisor - Reddit

https://www.reddit.com/r/docker/comments/1h0c6v7/standalone_docker_monitoring_native_metrics_vs/

75 How to monitor docker containers using cAdvisor, Prometheus and ...

https://www.reddit.com/r/docker/comments/l9di2n/how_to_monitor_docker_containers_using_cadvisor/